

## 23/04/22 Languages for PDA:

### ① Acceptance by final state:

$$L_F(M) = \{w \mid (q_0, w, z_0) \xrightarrow{*} (p, \epsilon, \gamma) \quad \begin{matrix} p \in Q_f \\ \gamma \in \Gamma^+ \end{matrix}$$

initial input      bottom state      final state  
 ↗                      ↘                      ↗

↳ can have elements in stack.

Eg:  $L = \{w \mid w \in (0/1)^*\}$

$$\delta(q_1, \epsilon, z) = \{(q_2, z)\}$$

↳ final state

### ② Acceptance by Empty stack:

$p \in Q$

$$L_E(M) = \{w \mid (q_0, w, z_0) \xrightarrow{*} (p, \epsilon, \epsilon)\}$$

empty stack

$$\delta(q_1, \epsilon, z) = \{(q_1, \epsilon)\}$$

↳ Empty stack

no need to go to final state

### ③ Acceptance by Reaching Final state & Empty stack:

$p \in F$

$$L_{FE}(M) = \{w \mid (q_0, w, z_0) \xrightarrow{*} (p, \epsilon, \epsilon)\}$$

$$\delta(q_1, \epsilon, z) = \{(q_2, \epsilon)\}$$

final state

↳ Empty stack

6)  $L = \{a^n b^m c^n \mid n, m \geq 1\}$

Construct a NPDA by reaching final state & emptying stack

$$L = \{abc, abbc, aabbcc, \dots\}$$

Pushing:

(i)  $\delta(q_0, a, z) = \{(q_0, az)\}$

(ii)  $\delta(q_0, a, a) = \{(q_0, aa)\}$

Shift:

(i)  $\delta(q_0, b, a) = \{(q_1, a)\}$

(ii)  $\delta(q_1, b, a) = \{(q_1, a)\}$

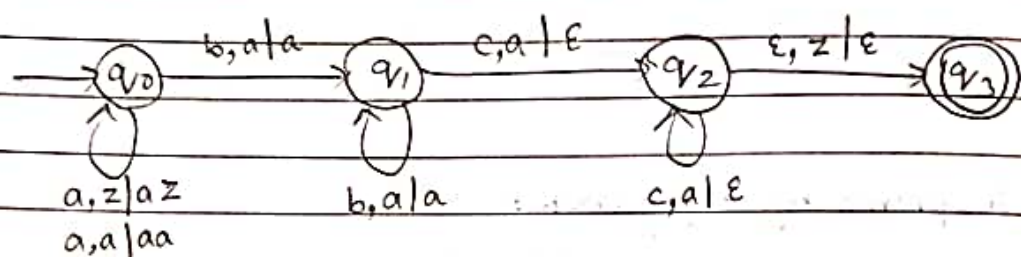
(iii)  $\delta(q_1, c, a) = \{(q_2, \epsilon)\}$

Begin:

$$i) \delta(q_2, c, a) = \{(q_2, \epsilon)\}$$

Final:

$$ii) \delta(q_2, \epsilon, z) = \{(q_3, \epsilon)\}$$



29/04/24 Equivalence of PDA (Unit-3):

$$CFG \iff PDA$$

If  $L$  is generated by a CFG, the same  $L$  can be generated/accepted by PDA & vice versa.

CFG to PDA:

$$\text{Step-1: } \delta(q_0, \epsilon, z) = \{(q_1, sz)\}$$

Push the  
start symbol

don't read  
any input

$$\text{Step-2: For all non-terminals 'A': } A \rightarrow \alpha$$

any NT (includes S)

$$A \rightarrow \alpha$$

read no input

$$\alpha = (NUT)^*$$

$$\delta(q_1, \epsilon, A) = \{(q_1, \alpha)\}$$

↳ Pop A & push  $\alpha$  to stack

$$\text{Step-3: } \forall \text{ Terminals 'a':}$$

$$\delta(q_1, a, a) = \{(q_1, \epsilon)\} \cdot \text{Pop } a$$

Step-4: Reaching final state:

$$\delta(q_1, \epsilon, z) = \{(q_2, z)\}$$

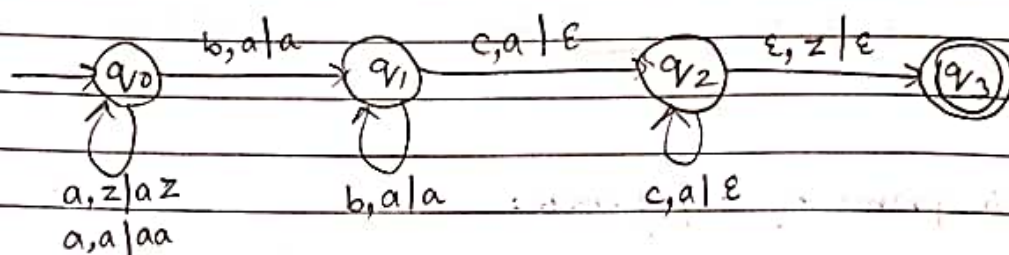
no more  
input symbols

Poping:

$$i) \delta(q_2, c, a) = \{(q_2, \epsilon)\}$$

Final:

$$ii) \delta(q_2, \epsilon, z) = \{(q_3, \epsilon)\}$$



29/04/24 Equivalence of PDA (Unit-3):

$$CFG \iff PDA$$

If  $L$  is generated by a CFG, the same  $L$  can be generated/accepted by PDA & vice versa.

CFG to PDA:

Step-1:  $\delta(q_0, \epsilon, z) = \{(q_1, sz)\}$

Push the  
start symbol

↓  
don't read  
any input

Step-2: For all non-terminals 'A':  $A \rightarrow \alpha$

↑  
read no input

any NT (includes S)

$$A \rightarrow \alpha$$

$$\alpha = (NT)^*$$

$$\delta(q_1, \epsilon, A) = \{(q_1, \alpha)\}$$

↑  
Pop A & push  $\alpha$  to stack

Step-3:  $\forall$  Terminals 'a':

$$\delta(q_1, a, a) = \{(q_1, \epsilon)\} \quad \text{Pop a}$$

Step-4: Reaching final state:

$$\delta(q_1, \epsilon, z) = \{(q_2, z)\}$$

↓  
no more  
input symbols

- a)  $S \rightarrow aSA \mid a$   
 $A \rightarrow bB$   
 $B \rightarrow b$

Construct a PDA  $\vdash$ :

validate aabb

$$\begin{aligned} S &\xRightarrow{1m} aSA & (S \rightarrow a) \\ &\xRightarrow{1m} aaA & (A \rightarrow bB) \\ &\xRightarrow{1m} aabb & (B \rightarrow b) \\ &\xRightarrow{1m} aabb \end{aligned}$$

1) Push S to stack.

$$\delta(q_0, \epsilon, z) = \{(q_1, SZ)\}$$

2) NT:  $S, A, B$

$$\delta(q_1, \epsilon, S) = \{(q_1, aSA), (q_1, a)\}$$

$$\delta(q_1, \epsilon, A) = \{(q_1, bB)\}$$

$$\delta(q_1, \epsilon, B) = \{(q_1, b)\}$$

3)  $T: a, b$

$$\delta(q_1, a, a) = \{(q_1, \epsilon)\}$$

$$\delta(q_1, b, b) = \{(q_1, \epsilon)\}$$

4) Final state:

$$\delta(q_1, \epsilon, z) = \{(q_2, z)\}$$

$$(q_0, \epsilon aabb, z)$$

$$\vdash (q_1, \epsilon aabb, SZ)$$

$$\vdash (q_1, aabb, aSAZ)$$

$$\vdash (q_1, \epsilon aabb, SAZ)$$

$$\vdash (q_1, abb, AAZ)$$

$$\vdash (q_1, bbb, AZ)$$

$$\vdash (q_1, bb, bBZ)$$

$$\vdash (q_1, b, BZ)$$

$$\vdash (q_1, \epsilon, bZ)$$

$$\vdash (q_1, \epsilon, Z)$$

$$\vdash (q_2, \epsilon, Z) \quad q_2 \in F$$

$\therefore$  aabb is accepted.

②  $S \rightarrow aSa \mid bSb \mid c$

Convert to PDA.

- Push S  $1) \delta(q_0, \epsilon, Z) = \{(q_1, SZ)\}$
- Push  $\{ 2) \delta(q_1, \epsilon, S) = \{(q_1, aSa), (q_1, bSb), (q_1, c)\}$
- Pop  $\{ 3) \delta(q_1, a, a) = \{(q_1, \epsilon)\}$
- 4)  $\delta(q_1, b, b) = \{(q_1, \epsilon)\}$
- 5)  $\delta(q_1, c, c) = \{(q_1, \epsilon)\}$
- Final 6)  $\delta(q_1, \epsilon, Z) = \{(q_2, Z)\}$

③  $S \rightarrow OS \mid A$

$A \rightarrow 1A0 \mid S \mid \epsilon$

- Push S  $1) \delta(q_0, \epsilon, Z) = \{(q_1, SZ)\}$
- Push  $\{ 2) \delta(q_1, \epsilon, S) = \{(q_1, OS), (q_1, A)\}$
- 3)  $\delta(q_1, \epsilon, A) = \{(q_1, 1A0), (q_1, S), (q_1, \epsilon)\}$
- Pop  $\{ 4) \delta(q_1, O, O) = \{(q_1, \epsilon)\}$
- 5)  $\delta(q_1, 1, 1) = \{(q_1, \epsilon)\}$
- Final 6)  $\delta(q_1, \epsilon, Z) = \{(q_2, Z)\}$

④  $A \rightarrow aABC \mid bB \mid a$

$B \rightarrow b$

$C \rightarrow a$

- Push  $\{ 1) \delta(q_0, \epsilon, Z) = \{(q_1, SZ)\}$
- 2)  $\delta(q_1, \epsilon, A) = \{(q_1, aABC), (q_1, bB), (q_1, a)\}$
- 3)  $\delta(q_1, \epsilon, B) = \{(q_1, b)\}$
- 4)  $\delta(q_1, \epsilon, C) = \{(q_1, a)\}$
- Pop  $\{ 5) \delta(q_1, a, a) = \{(q_1, \epsilon)\}$
- 6)  $\delta(q_1, b, b) = \{(q_1, \epsilon)\}$
- Final 7)  $\delta(q_1, \epsilon, Z) = \{(q_2, Z)\}$

deterministic PDA,

$$\delta(p, \alpha, \beta) = (q, \gamma)$$

Only one choice of move for any state  $p$ .

don't consider input symbols

\*  $\delta(p, \epsilon, \beta)$  is not empty then  $\delta(p, a, \beta)$  must be empty for  $\forall a \in \Sigma, p \in Q, \beta \in \Gamma$

① Construct NPDA for  $L = \{a^n b^n \mid n \geq 0\}$

Pushing:

$$\delta(q_0, a, z) = \{(q_0, az)\}$$

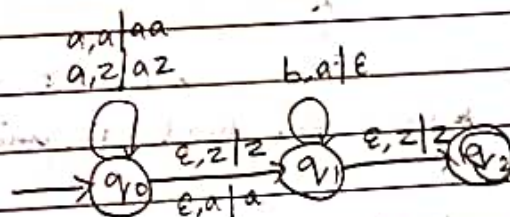
$$\delta(q_0, a, a) = \{(q_0, aa)\}$$

Shift:

$$\delta(q_0, \epsilon, z) = \{(q_1, z)\}$$

$$\delta(q_0, \epsilon, a) = \{(q_1, \tilde{a})\}$$

$b, a$  pop while shifting



Pop:

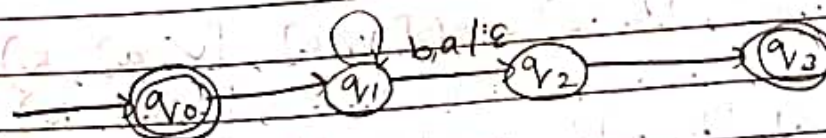
$$\delta(q_1, b, a) = \{(q_1, \epsilon)\}$$

Final:

$$\delta(q_1, \epsilon, z) = \{(q_2, z)\}$$

deterministic:

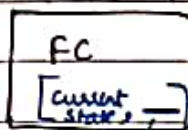
Make  $q_0$  as F to accept  $\epsilon$   
Pop  $a$  while shifting itself



## 02/05/2024 Programming techniques of Turing machines:

## ① Storage in Finite control:

\* Along with current state, FC remembers the read input/tape symbol.



$\in \Sigma \text{ or } \Gamma$

\* Eg:  $\delta([q_0, B], 0) = \delta([q_1, 0], 0, R)$

← remembers 0

## ② Multiple Tracks:

\* Multiple tracks of one way infinite tape.

|   |   |   |   |   |     |
|---|---|---|---|---|-----|
| B | 1 | 0 | 1 | . | .   |
| B | B | B | 1 | 0 | ..  |
| 1 | 0 | 1 | 0 | 1 | ... |

↑

FC

points to all 3 tracks.

\* Only one head / FC pointer.

## ③ Checking of symbols:

Eg:  $L = \{a^n b^n \mid n \geq 1\}$

|         |    |   |    |   |
|---------|----|---|----|---|
| Track 1 | ✓B | B | ✓B | B |
| Track 2 | a  | a | ✓b | b |

←

instead of replacing a, b by x, y,

same logic with storage for track 1

Q)  $L = \{a^n b^n \mid n \geq 1\}$

Forward:  $\delta([q_0, B], [B, a]) = ([q_1, a], [\checkmark, a], R)$

$\delta([q_1, a], [B, a]) = ([q_1, a], [B, a], R)$

$\delta([q_1, a], [B, b]) = ([q_2, B], [\checkmark, b], L)$

$\delta([q_1, a], [\checkmark, b]) = ([q_1, a], [\checkmark, b], R)$  so we need to remember.

Backward:  $\delta([q_2, B], [\checkmark, b]) = ([q_2, B], [\checkmark, b], L)$

$\delta([q_2, B], [B, a]) = ([q_2, B], [B, a], L)$

$\delta([q_2, B], [\checkmark, a]) = ([q_0, B], [\checkmark, a], R)$

$\delta([q_0, B], [\checkmark, b]) = ([q_3, B], [\checkmark, b], R)$

Terminate:  $\delta([q_3, B], [\checkmark, b]) = ([q_3, b], [\checkmark, b], R)$

$\delta([q_3, B], [B, B]) = ([q_1, B], [B, B], R)$

$$L = \{0^n 1^n 2^n \mid n \geq 1\}$$

classmate

Date

Page

Q)  $L = \{w \mid w^R \mid w \in (0/1)^*\}$

|   |   |   |   |   |
|---|---|---|---|---|
| ✓ | ✓ | ✓ | ✓ |   |
| B | B | B | B | B |
| 0 | 1 | 1 | 0 | B |

need  $\delta([q_0, B], [B, 0]) = ([q_1, 0], [\checkmark, 0], R)$

$\delta([q_1, 0], [B, 1]) = ([q_1, 0], [B, 1], R)$

$\delta([q_1, 0], [B, 0]) = ([q_1, 0], [B, 0], R)$

$\delta([q_1, 0], [\checkmark, 1]) = ([q_1, 0], [\checkmark, 1], R)$

$\delta([q_1, 0], [\checkmark, 0]) = ([q_1, 0], [\checkmark, 0], R)$

$\delta([q_1, 0], [B, B]) = ([q_2, 0], [B, B], L)$

$\delta([q_2, 0], [B, 0]) = ([q_3, B], [\checkmark, 0], L)$

ackward  $\delta([q_2, 0], [\checkmark, 0]) = ([q_2, 0], [\checkmark, 0], L)$

$\delta([q_2, 0], [\checkmark, 1]) = ([q_2, 0], [\checkmark, 1], L)$

$\delta([q_3, B], [\checkmark, 0]) = ([q_0, B], [\checkmark, 0], R)$

$\delta([q_3, B], [\checkmark, 1]) = ([q_3, B], [\checkmark, 1], L)$

$\delta([q_3, B], [B, 0]) = ([q_3, B], [B, 0], L)$

$\delta([q_3, B], [B, 1]) = ([q_3, B], [B, 1], L)$

03/08/24 (1) Subroutine: (called inside a machine)

Multiplication:

0010010000

copy 1 x no. of times to resultant

B 0 1 0 0 1 B B B B

X

→ 0

← X

→ X → 0

no more 0's ∴ replace X by 0's

B 0 1 0 0

←

→ B 1 X 0 ... 0 B

repeat

after B no 0's, terminate. (change everything to B till 2nd)

$$\delta(q_0, 0) = (q_6, B, R)$$

$$\delta(q_6, 0) = (q_6, 0, R)$$

$$\delta(q_6, 1) = (q_1, 1, R)$$

Subroutine for Copy:

| $\delta$ | 0             | 1             | X             | B             |
|----------|---------------|---------------|---------------|---------------|
| $q_1$    | $(q_2, X, R)$ | $(q_4, 1, L)$ |               |               |
| $q_2$    | $(q_2, 0, R)$ | $(q_2, 1, R)$ |               | $(q_2, 0, L)$ |
| $q_3$    | $(q_3, 0, L)$ | $(q_3, 1, L)$ | $(q_1, X, R)$ |               |
| $q_4$    |               | $(q_5, 1, R)$ | $(q_4, 0, L)$ |               |

Remaining transitions (lms)

Validate 2x2 output:

B 0 0 1 0 0 1 B

$q_0$  0 0 1 0 0 1 B

└ B  $q_6$  0 1 0 0 1

└ B 0  $q_6$  1 0 0 1

└ B 0 1  $q_1$  0 0 1

↑ subroutine

$\vdash B01Xq_201$  $\vdash B01X0q_21$  $\vdash B01X01q_2B$  $\vdash B01X0q_310$  $\vdash B01Xq_3010$  $\vdash B01q_3X010$  $\vdash B01Xq_1010$  $\vdash B01XXq_210$  $\vdash B01XX1q_20$  $\vdash B01XX10q_2B$  $\vdash B01XX1q_300$  $\vdash B01XXq_3100$  $\vdash B01Xq_3X100$  $\vdash B01XXq_1100$  $\vdash B01Xq_4X100$  $\vdash B01q_4X0100$  $\vdash B0q_4100100$  $\vdash B01q_500100$  $\vdash B0q_7100100$  $\vdash Bq_80100100$  $\vdash q_9B0100100$  $\vdash Bq_{10}0100100$  $\vdash BBq_6100100$  $\vdash BB1q_100100$  $\vdash BB1Xq_20100$  $\vdash BB1X0q_2100$  $\vdash BB1X01q_200$  $\vdash BB1X010q_20$  $\vdash BB1X0100q_2B$  $\vdash BB1X010q_30$  $\vdash BB1X01q_3000$  $\vdash BB1X0q_31000$  $\vdash BB1Xq_301000$  $\vdash BBq_3X01000$  $\vdash BB1Xq_101000$  $\vdash BB1XXq_21000$  $\vdash BB1XX1q_2000$  $\vdash BB1XX10q_200$  $\vdash BB1XX100q_20B$  $\vdash BB1XX1000q_2B$  $\vdash BB1XX10q_3000$  $\vdash BB1XX1q_30000$  $\vdash BB1XXq_310000$  $\vdash BB1Xq_3X10000$  $\vdash BB1XXq_110000$  $\vdash BB1Xq_4X10000$  $\vdash BB1q_4X010000$  $\vdash BBq_410010000$  $\vdash BB1q_50010000$  $\vdash BBq_710010000$  $\vdash Bq_8B10010000$  $\vdash BBq_{10}10010000$  $\vdash BBBq_{11}0010000$  $\vdash BBBBq_{11}010000$  $\vdash BBBBBq_{11}10000$  $\vdash BBBBBBq_{12}0000$

07/09/24 Extensions of Turing Machine:

(no error)

① Multiple tape:

\* Each tape has separate head.

(Input/Output/operation etc)



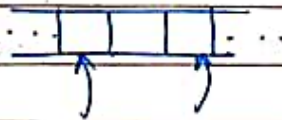
3 heads.

② 2-way infinite TM:



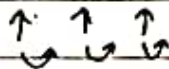
③ Multiple head:

\* One single 2 way infinite tape with multiple heads



3 heads or &gt;

1 a a b b c c

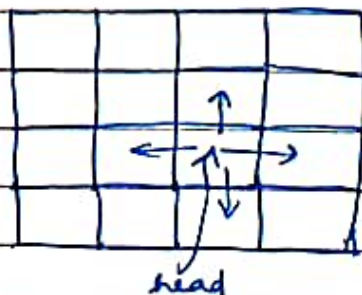


Terminate: 1st heads reads b

2nd head reads c

3rd head reads B

④ 2 D tape:



→; mov.

⑤ Random Access TM:

- \* <sup>Program</sup> Registers, Counters, etc. One way to tape, instructions.
- \* Evolution of microprocessor.

⑥ Non deterministic Turing Machine:

$$\delta: Q \times \Gamma \rightarrow 2^{Q \times \Gamma \times \{L, R\}}$$

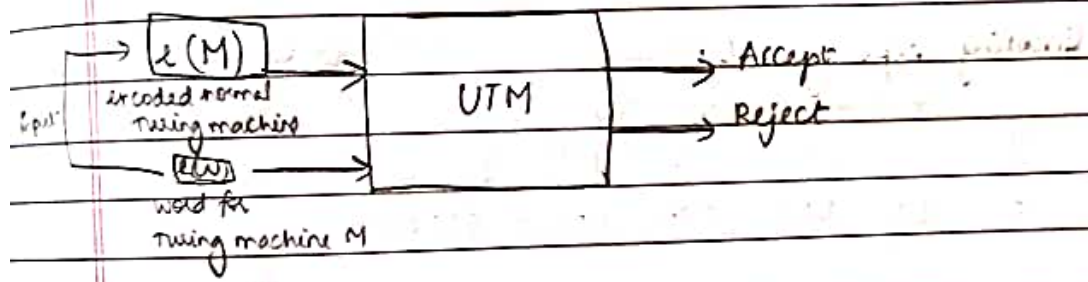
subset of states

15/24

Unit - 5

Universal Turing Machine:

- \* Works for any language.

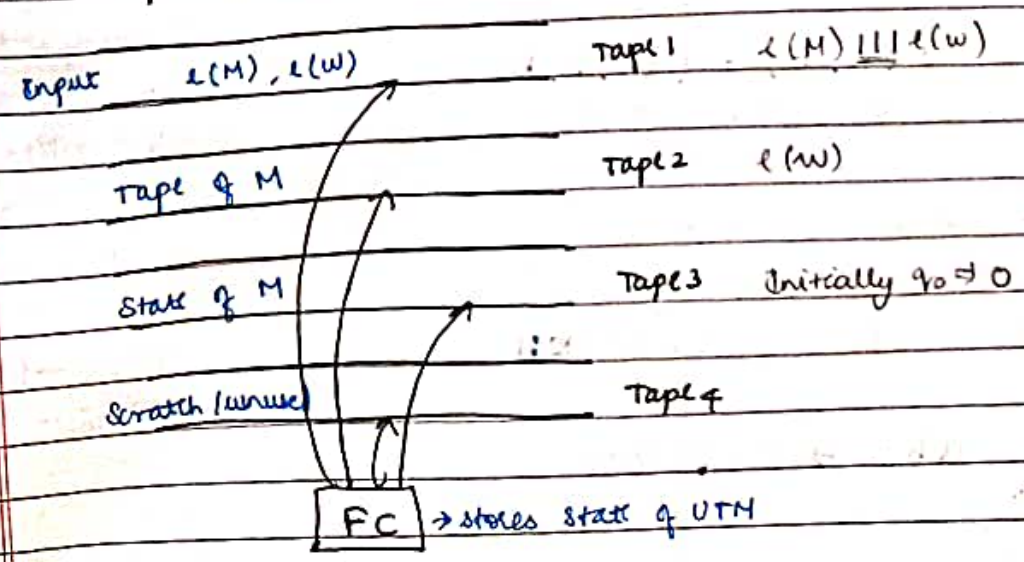


\* Machine in encoded form, word in encoded form is accepted if  $w$  is accepted by  $M$ , UTM rejects if  $w$  is rejected by  $M$ .

NOTE:

\* UTM is constructed using Multi Tape TM.

Multi Tape TM:



$L_u = \{ \langle M, w \rangle \mid M \text{ accepts } w \}$   
 halts if  $M$  accepts  $w$   
 outputs if  $M$  produces new word for  $w$   
 rejects if  $M$  rejects  $w$

UTM Machine:

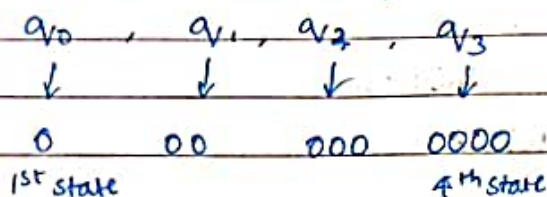
$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

$$\delta(q_i, t_s) = (n_s, t_s, L/R)$$

Unary form  
Encoding

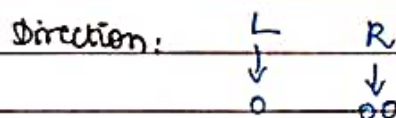
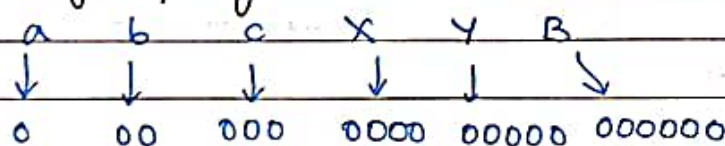
Encoding state:

Encode  $q_n$ ,  $n \rightarrow$  unary form i.e.  $(n+1)$  zeroes.



Encoding tape symbol:

Order the symbols.



Example:  $\delta(q_0, a) = (q_2, X, L)$



Combine the encodings by '1'

$$\begin{aligned} q_0 &\rightarrow 0 & q_2 &\rightarrow 000 & L &\rightarrow 0 \\ a &\rightarrow 0 & x &\rightarrow 0000 \end{aligned}$$

$$\Rightarrow 0 \mid 0 \mid 000 \mid 0000 \mid 0$$

$$\delta(q_1, b) = (q_0, b, R)$$

$$\Rightarrow 00 \mid 00 \mid 0 \mid 00 \mid 00$$



Combine all transitions by '11'.

$$M \Rightarrow \therefore 010100010000110010100100$$

Input word.

$$abc \Rightarrow 01001000$$

8) Encode given TM  
 working principle of UTM, example (2 transitions)

Combine  $M \times W$  by '111':

0101000100001011001010010011101001000

M W

$M: \delta(q_3, t_5) = (q_5, t_5, L/R)$  can be simplified as:

$\begin{matrix} i & j & k & l & m \\ 0 & 1 & 0 & 1 & 0 \end{matrix}$ 
 → move L/R in tape 2, & continue to search in tape 1  
 → replace 01 by 0<sup>1</sup> in tape 2  
 → tape 3 new content

∴ ① ⇒ 0<sup>1</sup>0<sup>1</sup>0<sup>3</sup>10<sup>4</sup>10<sup>1</sup>

② ⇒ 0<sup>2</sup>10<sup>2</sup>10<sup>1</sup>0<sup>2</sup>10<sup>2</sup>

tape 1:  $e(M) \Rightarrow 0^1 0^1 0^3 10^4 10^1 11 0^2 10^2 10^1 0^2 10^2$

tape 2:  $e(W) \Rightarrow 0^1 0^2 10^3$       (01) → modified to 0<sup>1</sup>

tape 3: (0<sup>i</sup>) = 0<sup>1</sup>      (∵ q<sub>0</sub>)

Based on content of tape 2, 3, tape 2 is modified by replacing symbol x in tape 1 is traversed L/R.

Undecidable Problem: REL, NREL  
 ↳ semidecidable (TM halts on accept)

REL: 1) Halting Problem (H.P.)  
 2) Post constraint Correspondence (P.C.P.)

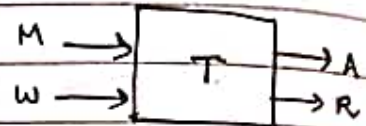
NREL: 2) Diagonalisation Problem (L.d)

① Halting Problem: (C program)  
 \*We can't tell / write a program to tell whether another problem halts / not.

language:  
 $Halt_m = \{ \langle m, w \rangle \mid m \text{ halts on } w \}$

Theorem:

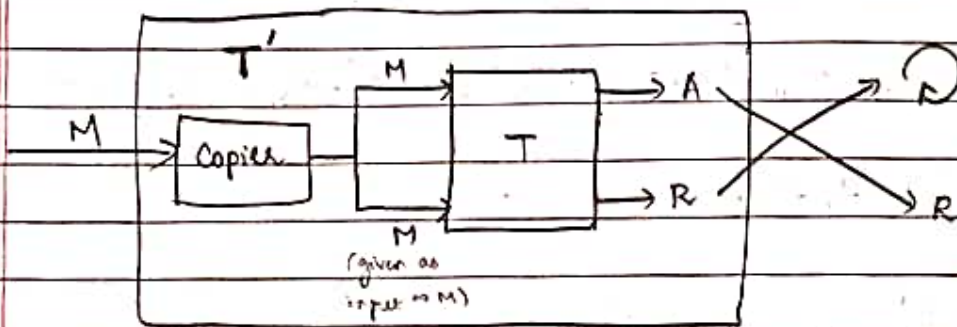
\* Halting problem is undecidable.



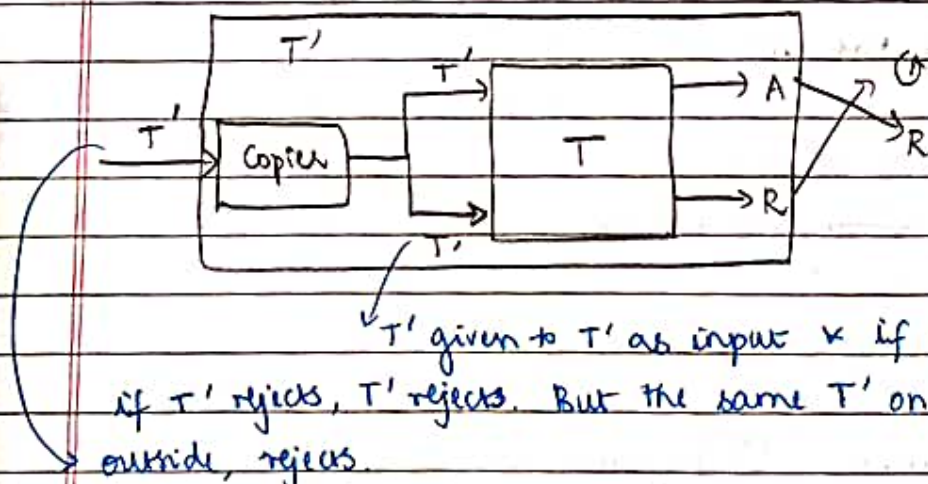
Proof: (By Contradiction)

T can't say that M always halts.

Assume that halting problem is decidable.  
Consider the above Turing machine T. (always halts)



Copy M to give M itself as input to M.  
Now, give T' itself as input to T'.



T' given to T' as input  $\times$  if it accepts, T accepts, if T' rejects, T' rejects. But the same T' on accepting T' outside, rejects.

$\therefore$  Contradiction occurs.

② Diagonalisation Language ( $L_d$ )

$$L_d = \{ TM_1, TM_2, \dots \}$$

encoded M, w

\* Write Turing machines in Canonical forms

$M_1, M_2, \dots, M_k$

words:  $\{\epsilon, 0, 1, 01, 10, \dots\}$

word  $1, 2, \dots$

accepted = 1

rejected = 0

instead of words give machines itself

| M \ W | $M_1$<br>$w_1$ | $M_2$<br>$w_2$ | $M_3$<br>$w_3$ | $M_4$<br>$w_4$ | $M_5$<br>$w_5$ | $M_6$<br>$w_6$ | ...                                 |
|-------|----------------|----------------|----------------|----------------|----------------|----------------|-------------------------------------|
| $M_1$ | 1              | 0              | 1              | 0              | 1              | 0              | $\rightarrow w_6$ rejected by $M_1$ |
| $M_2$ | 0              | 0              | 1              | 0              | 0              | 0              |                                     |
| $M_3$ | 0              | 0              | 0              | 1              | 1              | 0              |                                     |
| $M_4$ | 1              | 0              | 1              | 0              | 1              | 1              |                                     |
| $M_5$ | 1              | 1              | 1              | 1              | 1              | 1              |                                     |
| $M_6$ |                |                |                |                |                |                |                                     |

$\rightarrow$  machine  $M_1, M_5$  accepts itself  
 $M_2, 3, 4$  rejects itself

$L_d \rightarrow$  select machines/words that are rejected by Machine strings

$L_d = \{w_i \mid w_i \notin L(M_i)\}$   
 $L_d = \{w_i \mid T[i, i] = 0\}$

$\rightarrow$  Non Recursively enumerable language. (No TM exists)

Proof:

\* Assume  $M_k$  exists.

(actually doesn't exist)

$L(M_k) = L_d$

1) If  $w \in L(M_k)$ ,  $T[k, k] = 1 \Rightarrow w \notin L_d$  (machine exists)

2) If  $w \notin L(M_k)$ ,  $T[k, k] = 0 \Rightarrow w \in L_d$  (machine doesn't exist)

$\therefore$  By contradictions,  $M_k$  doesn't exist.  
(our assumptions)

③ Post Correspondence Problem (PCP):

\* Two arrays A, B correspondence two words & find solution

| Pair no. | A     | B     |
|----------|-------|-------|
| 1        | $w_1$ | $x_1$ |
| 2        | $w_2$ | $x_2$ |
| 3        | $w_3$ | $x_3$ |
| ...      | ...   | ...   |

Select pair  $(w_i, x_i)$

Can repeat/omit pairs

Example 1

|   | A   | B   |
|---|-----|-----|
| 1 | 100 | 001 |
| 2 | 11  | 111 |
| 3 | 111 | 11  |

can't match

100 001  
x

$$\begin{array}{r} 2: \quad 111 \\ \underline{\quad 111 \quad} \\ \quad 1 \end{array}$$

so lets take 1

$$1: \quad \begin{array}{r} 100 \\ \times 00 \\ \hline 000 \end{array}$$

2 1 3  
A: 11|100|111  
B: 11|100|111

(2 1 3) ans

②

| A    | B   |
|------|-----|
| 00   | 0   |
| 001  | 11  |
| 1000 | 011 |

00 x 001 00 (1000  
000 x 11 01  
6x

No solution.

Modified PCP: (MPCP → Undecidable)



First correspondence pair should always be 1.

③

| A    | B   |
|------|-----|
| 11   | 1   |
| 1    | 111 |
| 0111 | 10  |
| 10   | 0   |

11110  
11110

(1 2 4)

~~1011111110~~  
~~1101110~~

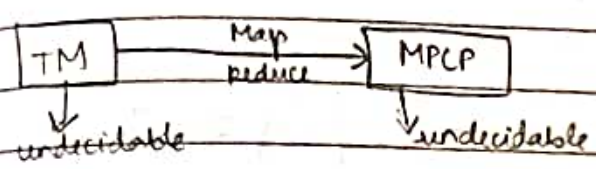
(1 3 2 2 4)

1101111110  
1101111110

Proof: (MPCP is undecidable)

Map a TM to MPCP

∴ As TM is undecidable, MPCP is also undecidable



05/24

Unit - 3

Equivalence of PDA:

CFG  $\Rightarrow$  PDA

$N \rightarrow (NUT)^*$   $\delta(q_0, q_0, q_0)$   
 $= (rs, rs)$

PDA:  $\rightarrow \{0,1\}$

$M = (Q, \Sigma, \Gamma, \delta, q_0, Z)$   
 $\hookrightarrow \{q, r\} \hookrightarrow \{x, z\}$

CFG:

$G = (N, T, P, S)$   $T = \{0,1\}$   
 $S \rightarrow$  start symbol.

Given:

- $\delta(q, 0, Z) = (q, XZ)$
- $\delta(q, 0, X) = (q, XX)$
- $\delta(q, 1, X) = (r, \epsilon)$
- $\delta(r, 1, X) = (r, \epsilon)$
- $\delta(r, \epsilon, Z) = (r, \epsilon)$

Step-1:

Non Terminal:

[state, stack symbol, state]

- 1) S
- 2) [q Z q]
- 3) [q Z r]
- 4) [q Z q]
- 5) [r Z r]
- 6) [q X q]
- 7) [q X r]
- 8) [r X q]
- 9) [r X r]

(∵ 2 states, 4 combinations)

(2 stack symbols  $\rightarrow 4 \times 2 = 8$ )

Step-2:

Write the productions:-

$$(i) \boxed{S \rightarrow [ \underset{\downarrow q}{\text{start state}}, \underset{\downarrow z}{\text{start stack symbol}}, \overset{\downarrow \{q, r\}}{\text{all states}} ]}$$

$$\therefore S \rightarrow [qzq]$$

$$S \rightarrow [qzr]$$

$$\therefore S \rightarrow [qzq] \mid [qzr]$$

$$(ii) \delta(\text{state}, \overset{\text{I/P}}{\text{symbol}}, \overset{\text{stack}}{\text{symbol}}) = (\text{next state}, \epsilon) \quad (\text{Pop}):$$

$$(\delta(\delta, a, y) = (t, \epsilon))$$

$$\Rightarrow \boxed{[\delta y t] \rightarrow a}$$

$$\therefore [qxr] \rightarrow \epsilon$$

$$\delta(q, x, r) = (r, \epsilon)$$

$$[ryr] \rightarrow \epsilon$$

$$[rzt] \rightarrow \epsilon$$

$$(iii) \delta(\text{state}, \overset{\text{I/P}}{\text{symbol}}, \overset{\text{stack}}{\text{symbol}}) = (\text{next state}, \text{symbols}) \quad (\text{Push}):$$

$$\delta(s, a, y) = (t, y, y_2, y_3, \dots, y_n)$$

$$\Rightarrow [s y s_k] \rightarrow a [t y, \delta_1] [\delta_1 y_2 \delta_2]$$

$\hookrightarrow$   $\forall$  states in PDA

$k \rightarrow$  total no. of states in PDA

$$\dots [s_{k-1} y_n s_k]$$

$$\therefore \delta(q, a, z) = (q, xz)$$

$$\boxed{[\delta y s_2] \rightarrow a [t y, s_1] [s_1 y_2 s_2]}$$

(Push 2 stack symbols)

$$[qz s_2] \rightarrow \epsilon [qxs_1] [\delta, z s_2]$$

$$\text{let } \delta_1 = q, \delta_2 = q:$$

$$[qzq] \rightarrow \epsilon [qxsq] [qzq]$$

$$\delta_1 = r, \delta_2 = q:$$

$$[qzq] \rightarrow \epsilon [qxr] [rztq]$$

$$S_1 = q, S_2 = r:$$

$$[qz\theta] \rightarrow o[qxq][qzr]$$

$$S_1 = r, S_2 = r:$$

$$[qzr] \rightarrow o[qxr][rzr]$$

$$\therefore [qzq] \rightarrow o[qxq][qzq] \mid o[qx\cancel{r}][\cancel{\theta}zq]$$

$$[qzr] \rightarrow o[qxq][qzr] \mid o[qxr][rzr]$$

$$\textcircled{2} \therefore \delta(q, o, x) = (q, xx)$$

$$S_1 \rightarrow q, S_2 \rightarrow q: [qxq] \rightarrow o[qxq][qxq]$$

$$S_1 \rightarrow q, S_2 \rightarrow r: [qxr] \rightarrow o[qxq][qxr]$$

$$S_1 \rightarrow r, S_2 \rightarrow q: [qxq] \rightarrow o[qxr][rxq]$$

$$S_1 \rightarrow r, S_2 \rightarrow r: [qxr] \rightarrow o[qxr][rxr]$$

Productions:

$$1) S \rightarrow A \mid B$$

$$2) [qxq] \rightarrow o[qxq][qxq] \mid o[qx\cancel{r}][\cancel{\theta}zq]$$

$$3) [qzr] \rightarrow o[qxq][qzr] \mid o[qxr][rzr]$$

$$4) [r\cancel{z}q]$$

$$5) [rzr] \rightarrow \epsilon$$

$$6) [qxq] \rightarrow o[qxq][qxq] \mid o[qx\cancel{r}][\cancel{\theta}zq]$$

$$7) [qxr] \rightarrow 1 \mid o[qxq][qxr] \mid o[qxr][rxr]$$

$$8) [rxq]$$

$$9) [rxr] \rightarrow 1$$

Simplification:

Remove NT without productions.

Remove productions with removed NTs.

Replace the productions as A, B, ...

$$S \rightarrow A \mid B$$

$$A \rightarrow oCA$$

$$B \rightarrow oCB \mid oDE$$

$$E \rightarrow \epsilon$$

$$C \rightarrow oCC$$

$$D \rightarrow 1 \mid oCD \mid oDF$$

$$F \rightarrow 1$$